CIS 3300 · THE SPEEDRUN LEADERBOARD

Checkpoint 3

Putting Runs in Order

Starts from: SpeedrunCheckpoint2. **Ends at:** SpeedrunCheckpoint3. **Prepares you for:** Lab W14, Part 2: putting readings in order, and the generic class.

At the end of Checkpoint 2, the program kept every good run, but in the order the runs were submitted. A leaderboard ranks them. In this checkpoint, the `Run` record learns its *natural order*, fastest first, so Java can sort runs for you. Then we write a *generic* class, `Leaderboard`, that keeps the best few entries of any type that can be put in order, and use it to show the top three Celeste runs.

Before you start, see the *Getting Started* guide for how to import a project into Eclipse and for the rules the leaderboard uses for each field.

A note about this guide. I developed this guide with help from AI, which drafted much of the code, data, and explanations, and I review and revise them. Every program was compiled and run, and every output shown is real. If something looks wrong, does not match what you see in Eclipse, or contradicts the book or the notes, tell me. This guide shows how to build one program. It does not replace the notes, the assigned chapters, or the recorded lectures.

Step 1: Background: the Comparable interface

To sort objects, Java needs to know which of two objects comes first. A class tells Java its natural order by implementing the `Comparable` interface, which has one method:

```
public interface Comparable<T> {
    int compareTo(T other);
}
```

`a.compareTo(b)` returns a negative number if `a` comes before `b`, zero if they are tied, and a positive number if `a` comes after `b`. Only the sign matters, not the size of the number. Many classes you already use implement it: `String`, `Integer`, `Double`, and the two `java.time` classes in `Run`, `Duration` and `LocalDate`. So you can compare two times or two dates by calling their `compareTo`.

Two rules of thumb:

- **Compare the most important field first,** and move on to the next field only when the first one is tied.
- **Do not subtract to compare.** `return a - b;` looks tempting for numbers, but it can overflow, and it does not work at all for objects such as times and dates. Call the field's own `compareTo`, or `Integer.compare(a, b)` and `Double.compare(a, b)` for primitive values.

Step 2: Make Run comparable

Open `Run.java`. Add `implements Comparable<Run>` to the end of the record's header, and add a `compareTo` method below the compact constructor:

```
public record Run(String player, String game, String category, Duration time,
        LocalDate date, boolean verified) implements Comparable<Run> {

    // ... the compact constructor, exactly as before ...

    @Override
    public int compareTo(Run other) {
        int byTime = time.compareTo(other.time);
        if (byTime != 0) {
            return byTime;
        }
        int byDate = date.compareTo(other.date);
        if (byDate != 0) {
            return byDate;
        }
        return player.compareTo(other.player);
    }
```

- **The time decides first.** A shorter `Duration` comes first, so the fastest run is first.
- **Equal times go to the earlier date.** On a real leaderboard, the first player to reach a time holds it; a later run with the same time ties it but does not take the place.
- **Equal times on the same date go by player name,** so the order is always the same from one run of the program to the next.
- **A record can implement an interface,** like any class. It cannot extend a class, because every record already extends `java.lang.Record`.

Step 3: Sort one category

To see the order work, sort the Celeste Any% runs. In `main`, right after the line that logs `"Loaded ..."`, add the code below. Add imports for `java.util.Collections` and `speedrun.RunTime`.

```
List<Run> celeste = new ArrayList<>();
for (Run run : runs) {
    if (run.game().equals("Celeste") && run.category().equals("Any%")) {
        celeste.add(run);
    }
}
Collections.sort(celeste);
for (Run run : celeste) {
    System.out.println(RunTime.format(run.time()) + "  " + run.date() + "  " + run.
    player());
}
```

Run the program. Below the log messages from Checkpoint 2, it prints:

```
26:41.6  2026-02-14  Kestrel
26:41.6  2026-05-03  Moss
27:17.7  2026-05-20  Vela
28:05.1  2026-03-10  GrapeSoda
29:39.2  2026-08-02  Lumen
30:16.7  2026-01-31  NovaByte
30:30.4  2026-06-15  Tangent
30:48.7  2026-07-13  Lumen
```

`Collections.sort` called `compareTo` to decide the order; you never wrote a sorting loop. Kestrel and Moss have the same time, and Kestrel ranks first because that run was earlier. In Eclipse, the red log messages and these lines go to different streams, so they can appear in a slightly different order. Keep this code for now; Step 6 replaces it.

Step 4: Background: why a generic class

A leaderboard does one job: it keeps the best few entries, in order, and drops the rest. Nothing about that job depends on what the entries are. The same code could keep the three fastest runs, the three earliest dates, or the three highest scores. A *generic* class writes that code once, with a *type parameter*, `T`, standing for the type of entry. Whoever creates a leaderboard chooses the type: `Leaderboard<Run>`, `Leaderboard<LocalDate>`, and so on.

The class needs one promise from `T`: that its values can be compared, because it calls `compareTo` to decide where each entry goes. That promise is the *bound* in the class's header:

```
public class Leaderboard<T extends Comparable<? super T>>
```

Read it as "`T` is a type that can be compared with other `T` values." The `? super T` part is there for types such as `LocalDate`, whose `compareTo` is declared in an interface above it rather than in `LocalDate` itself. With the simpler bound `T extends Comparable<T>`, a `Leaderboard<LocalDate>` does not compile:

```
error: type argument LocalDate is not within bounds of type-variable T
```

Step 5: Write the Leaderboard class

In the `speedrun` package, create a new class named `Leaderboard`:

```
package speedrun;

import java.util.ArrayList;
import java.util.List;
import java.util.Objects;

public class Leaderboard<T extends Comparable<? super T>> {
    private final int capacity;
    private final List<T> entries = new ArrayList<>();

    public Leaderboard(int capacity) {
        if (capacity < 1) {
```

```
            throw new IllegalArgumentException(
                    "capacity must be at least 1, but was " + capacity);
        }
        this.capacity = capacity;
    }

    public boolean add(T entry) {
        Objects.requireNonNull(entry, "entry is required");
        int position = 0;
        while (position < entries.size()
                && entries.get(position).compareTo(entry) <= 0) {
            position++;
        }
        if (position >= capacity) {
            return false;
        }
        entries.add(position, entry);
        if (entries.size() > capacity) {
            entries.remove(entries.size() - 1);
        }
        return true;
    }

    public T getLeader() {
        if (entries.isEmpty()) {
            throw new IllegalStateException("the leaderboard is empty");
        }
        return entries.get(0);
    }

    public List<T> getEntries() {
        return List.copyOf(entries);
    }

    public int size() {
        return entries.size();
    }

    public int getCapacity() {
        return capacity;
    }
}
```

- **`T` is used like any type** inside the class: `List<T>` for the field, `T entry` for a parameter, `T` as a return type. When you create a `Leaderboard<Run>`, every `T` means `Run`.
- **`add` finds the new entry's place.** It walks down the board while the entry already there is not worse than the new one, which is what `compareTo(entry) <= 0` tests. If that place is past the last place on the board, the entry is not good enough, and `add` returns `false`. Otherwise it inserts the entry, and if the board is now too long, drops the last entry.
- **Ties keep the first entry ahead.** Because the test is `<= 0`, a new entry equal to one already on the board goes after it.

- **The constructor and `getLeader` throw standard exceptions,** as in the exceptions notes: `IllegalArgumentException` for a bad argument, and `IllegalStateException` when the board is in no state to answer, because it is empty:

```
Exception in thread "main" java.lang.IllegalStateException: the leaderboard is empty
```

- **`getEntries` returns a copy** that cannot be changed, so no code outside the class can add an entry in the wrong place or make the board too long.

Step 6: Use the leaderboard

Replace the code you added in Step 3 with a `Leaderboard<Run>` that keeps the top three Celeste runs. Replace the import of `java.util.Collections` with `speedrun.Leaderboard`.

```
Leaderboard<Run> celeste = new Leaderboard<>(3);
for (Run run : runs) {
    if (run.game().equals("Celeste") && run.category().equals("Any%")) {
        celeste.add(run);
    }
}
printBoard("Celeste, Any%", celeste);
```

Add this method to the class, below `main`:

```
private static void printBoard(String title, Leaderboard<Run> board) {
    System.out.println(title + " (top " + board.getCapacity() + ")");
    List<Run> entries = board.getEntries();
    for (int i = 0; i < entries.size(); i++) {
        Run run = entries.get(i);
        System.out.printf("%2d. %-10s %10s  %s%n", i + 1, run.player(),
                RunTime.format(run.time()), run.date());
    }
}
```

Run the program. Below the log messages, it prints:

```
Celeste, Any% (top 3)
 1. Kestrel       26:41.6  2026-02-14
 2. Moss          26:41.6  2026-05-03
 3. Vela          27:17.7  2026-05-20
```

The loop added all eight Celeste runs, and the leaderboard kept only the best three, in order. Nothing had to be sorted first.

Step 7: The same class, a different type

Add these lines at the end of `main`'s `try` block, with `import java.time.LocalDate;` at the top:

```
Leaderboard<LocalDate> earliest = new Leaderboard<>(3);
earliest.add(LocalDate.of(2026, 2, 14));
```

```
earliest.add(LocalDate.of(2025, 9, 3));
earliest.add(LocalDate.of(2026, 5, 3));
earliest.add(LocalDate.of(2025, 11, 15));
System.out.println(earliest.getEntries());
System.out.println("Leader: " + earliest.getLeader());
```

```
[2025-09-03, 2025-11-15, 2026-02-14]
Leader: 2025-09-03
```

The same class, unchanged, now keeps the three earliest dates, because a date's natural order is earliest first. The compiler also checks the type for you. Try adding a line that calls `earliest.add("Moss")`, or creating a `Leaderboard<Object>`, which cannot be compared:

```
error: incompatible types: String cannot be converted to LocalDate
error: type argument Object is not within bounds of type-variable T
```

Both mistakes are caught before the program ever runs. Delete the lines from this step before going on. Your project should now match SpeedrunCheckpoint3.

Check yourself

1. Kestrel and Moss both ran Celeste Any% in 26:41.6. Which part of `compareTo` decides who ranks first, and why is that the right rule for a leaderboard?
2. Is `kestrel.compareTo(vela)` negative, zero, or positive? (Kestrel ran 26:41.6, and Vela ran 27:17.7.)
3. Why does `Leaderboard` need a bound on `T`? What would go wrong in `add` without it?
4. The loop in `add` tests `compareTo(entry) <= 0`. What would change if it tested `< 0` instead?
5. Why does `getEntries` return `List.copyOf(entries)` instead of `entries`?

Answers

1. The times are equal, so the date comparison decides, and Kestrel's run on 2026-02-14 comes before Moss's on 2026-05-03. That matches how leaderboards work: the first player to reach a time holds it.
2. Negative. Kestrel's time is shorter, so Kestrel comes first, and the dates are never compared.
3. `add` calls `compareTo` on the entries. Without the bound, `T` could be any type, including one with no `compareTo`, so the compiler would reject that call. The bound promises every `T` has one.
4. A new entry equal to one already on the board would go ahead of it, so ties would go to the entry added last instead of the one that got there first.
5. Returning the list itself would let any code add, remove, or reorder entries, breaking the promise that the board is always in order and never too long. A copy that cannot be changed keeps the board in charge of its own entries.